



LIGER KERNEL

曾駿馳 Chun-Chih Tseng : Maintainer in Liger-Kernel

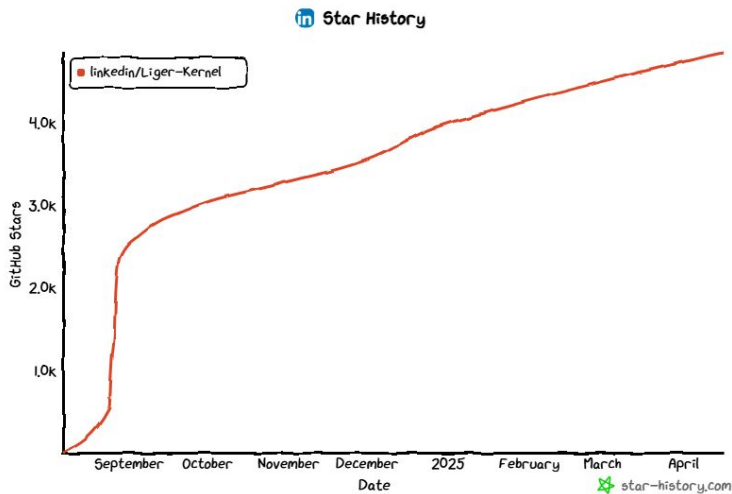
劉立行 Li-Hsing Liu : Maintainer in Liger-Kernel

Outline

- Introduction to Liger Kernel
- Inefficiencies in LLM Training
- Optimizations in Liger Kernel
 - Kernel Fusion, In-place tensor reuses, Gradient-in-forward & Chunking
 - Case Study: Fused Linear Cross Entropy
- Chunked Loss – Post Training Kernels
- Testing in Liger Kernel
 - Unit test
 - Convergence test
- Roadmap
- Q & A

Liger Kernel

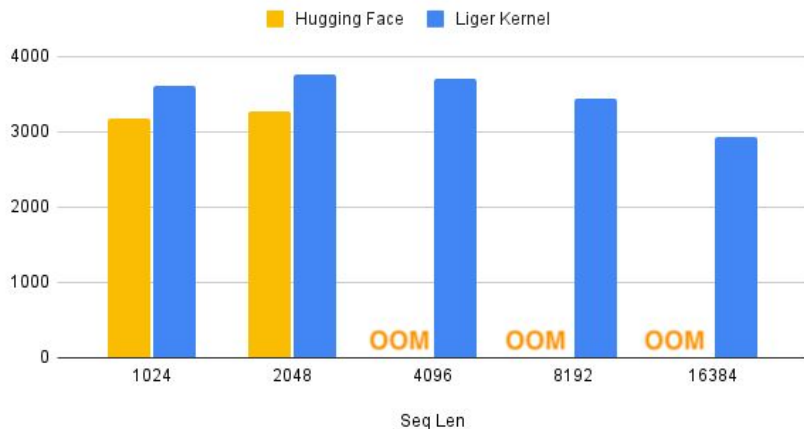
- One line change to make LLM Training 20% faster and 60% less memory
- 4900+ stars, 300+ forks, 70+ contributors, 2M downloads
- Adopted by NVIDIA, HuggingFace, Axolotl, Llama-factory, ...
- Sponsored by Glows.ai, AMD, Intel, ...



Liger Kernel

enabling longer context lengths, larger batch sizes, and massive vocabularies

Throughput (tokens/sec)



Peak Reserved Memory (MB)



- Benchmark conditions: LLaMA 3-8B, Batch Size = 8, Data Type = bf16, Optimizer = AdamW, Gradient Checkpointing = True, Distributed Strategy = FSDP1 on 8 A100s.
- Hugging Face models start to OOM at a 4K context length, whereas Hugging Face + Liger Kernel scales up to 16K.

Liger Kernel – Key Features

- Ease of use
 - Patching with one line of code
 - Composing your own model with Liger Kernel Modules
- Lightweight
 - Minimal dependencies – only Torch and Triton
- Multi-GPU supported
 - Compatible with multi-GPU setups (PyTorch FSDP, DeepSpeed, DDP, etc.)
- Trainer Framework Integration
 - [Axolotl](#), [LLaMa-Factory](#), [SFTTrainer](#), [Hugging Face Trainer](#), [SWIFT](#), [oumi](#)

Liger Kernel – API Usage

Use AutoLigerKernelForCausalLM



```
from liger_kernel.transformers import AutoLigerKernelForCausalLM

# This AutoModel wrapper class automatically monkey-patches the
# model with the optimized Liger kernels if the model is supported.
model = AutoLigerKernelForCausalLM.from_pretrained("path/to/some/model")
```

Liger Kernel – API Usage

Apply Model-Specific Patching APIs

```
import transformers
from liger_kernel.transformers import apply_liger_kernel_to_llama

# 1a. Adding this line automatically monkey-patches the model with the
# optimized Liger kernels
apply_liger_kernel_to_llama()

# 1b. You could alternatively specify exactly which kernels are applied
apply_liger_kernel_to_llama(
    rope=True,
    swiglu=True,
    cross_entropy=True,
    fused_linear_cross_entropy=False,
    rms_norm=False
)

# 2. Instantiate patched model
model = transformers.AutoModelForCausalLM("path/to/llama/model")
```

Liger Kernel – API Usage

Compose Your Own Model



```
from liger_kernel.transformers import LigerFusedLinearCrossEntropyLoss
import torch.nn as nn
import torch

model = nn.Linear(128, 256).cuda()

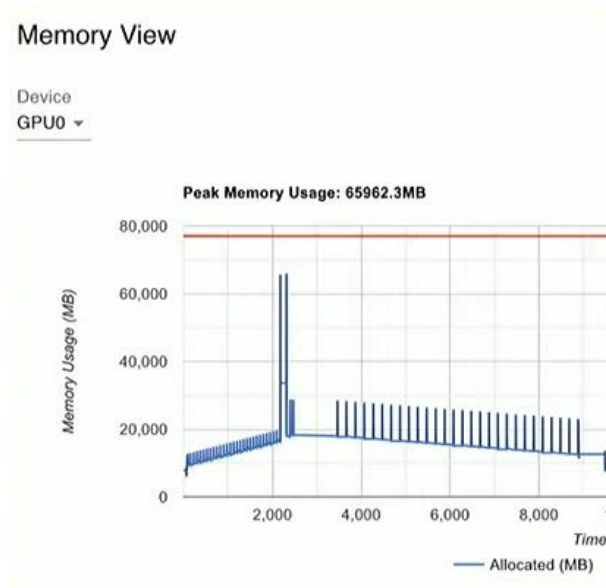
# fuses linear + cross entropy layers together and performs chunk-by-chunk
# computation to reduce memory
loss_fn = LigerFusedLinearCrossEntropyLoss()

input = torch.randn(4, 128, requires_grad=True, device="cuda")
target = torch.randint(256, (4, ), device="cuda")

loss = loss_fn(model.weight, input, target)
loss.backward()
```

Bottleneck in LLM Training

- Memory spike from logits
- Vocabulary size
 - Llama2: 32,000
 - Llama3: 128,256
 - Qwen2: 151936
 - Gemma3: 262,208



Bottleneck in LLM Training

- Quick math

- `batch_size = 4`
- `seq_length = 4096`
- `vocab_size = 128k`
- `dtype_size = 4` (casting to float32)

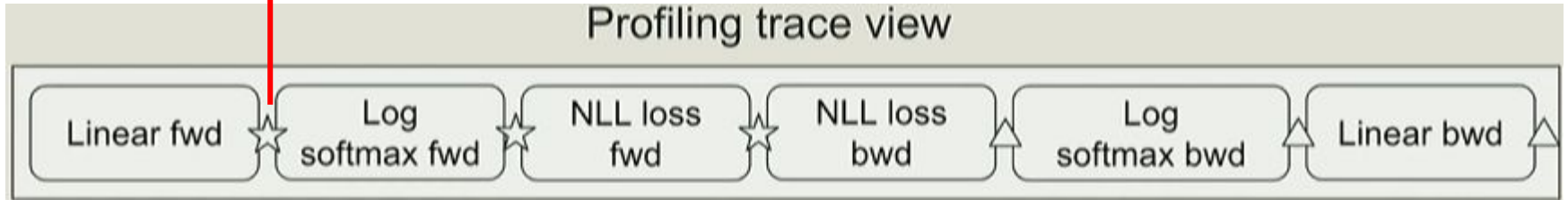
- Size of logits

`= batch_size * seq_length * vocab_size * dtype_size`

`= 4 * 4096 * 128k * 4 = 8 GB`

Optimization - Gradient-in-forward

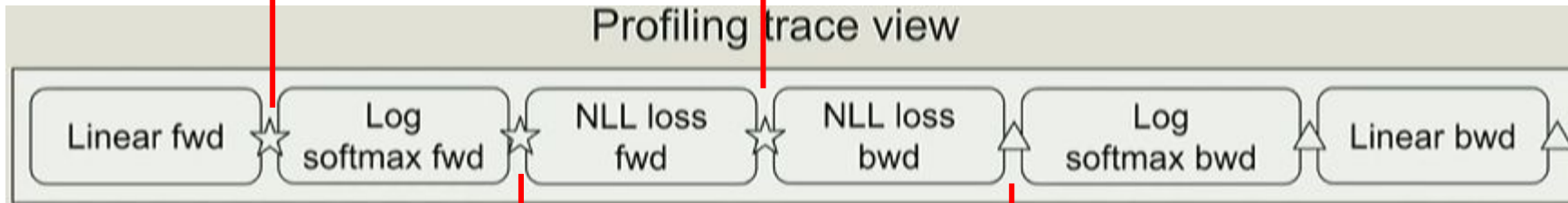
Size of logits ☆ = 8 GB



Optimization - Gradient-in-forward

Size of logits ☆ = 8 GB

Loss (neglectable)



LogSoftmax 8GB

Gradients of LogSoftmax 8GB

= 24 GB

- batch_size = 4
- seq_length = 4096
- vocab_size = 128
- dtype_size = 4 (casting to float32)

Bottleneck in LLM Training

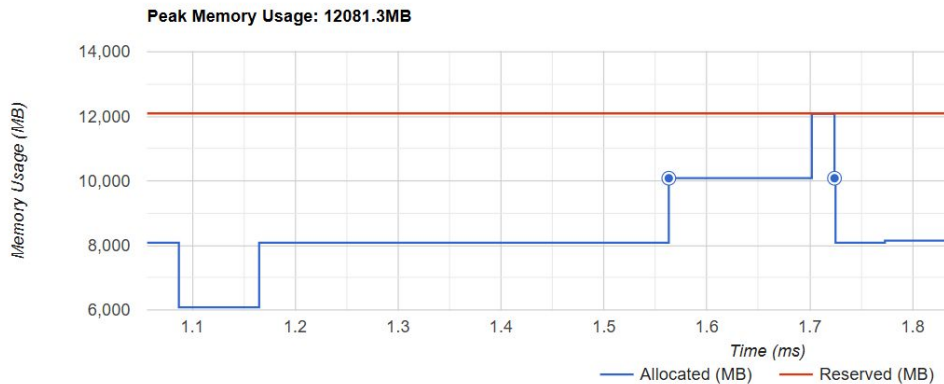
- Activations not including
 - Model's parameters
 - Model's gradients
 - Optimizer states
 - Scaling challenges: Longer context & larger vocab
- Tradeoff between memory and performance
 - CPU offloading
 - Gradient checkpointing
 - Gradient Accumulations

Bottleneck in LLM Training

- Simple Experiment
 - batch_size = 1, seq_length = 4096, vocab_size = 128, dtype_size = 4 (casting to float32)
- Size of logits = 2 GB

Memory View

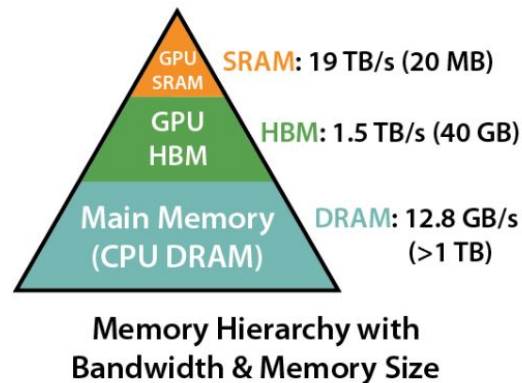
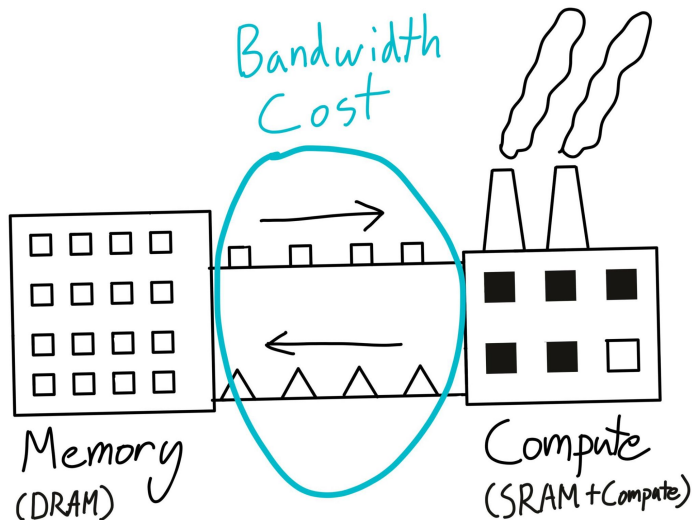
Device
GPU0 ▾



Optimizations in Liger Kernel – GPU Performance 101

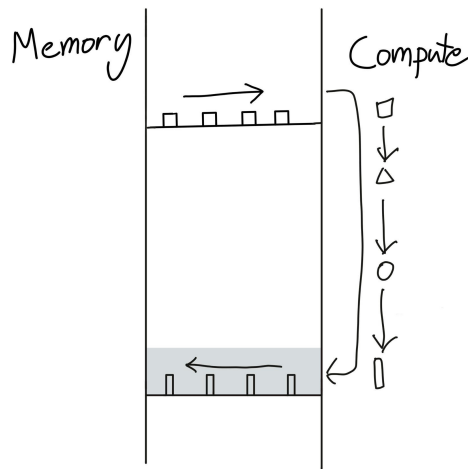
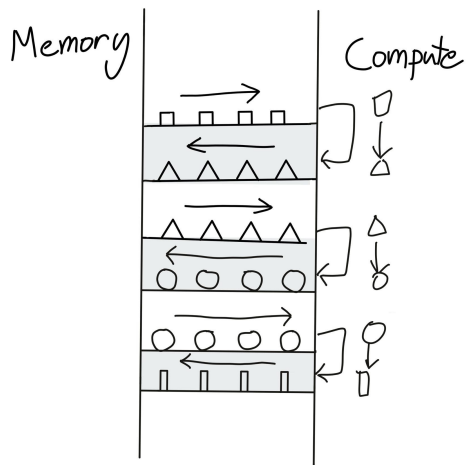
- Overhead time: dispatching kernels, sanity checks, ...
- Communication time: DRAM $\longleftrightarrow$ SRAM
- Computation time: torch.mm, ...

https://horace.io/brrr_intro.html

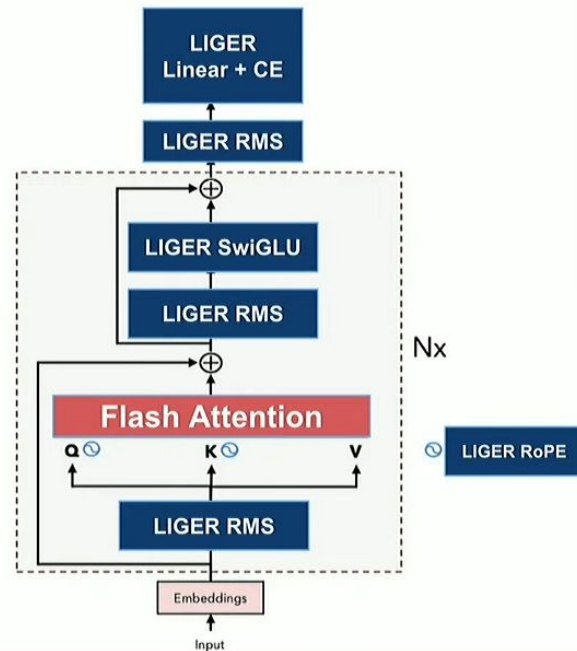
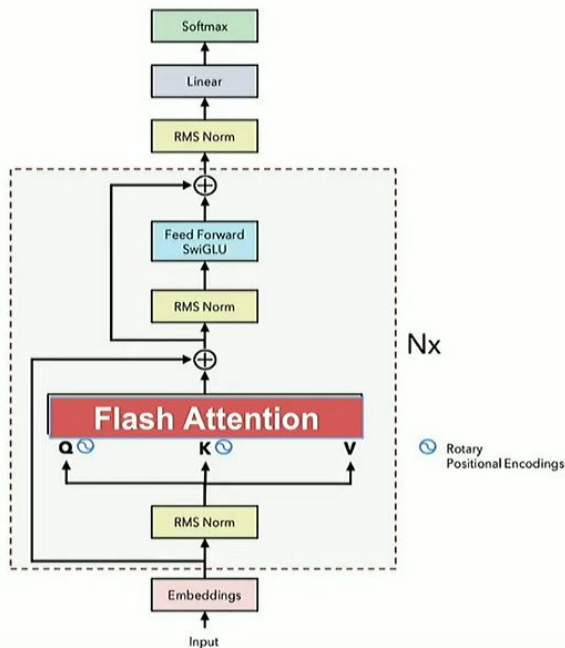


Optimizations in Liger Kernel – Kernel Fusion

- Memory access is expensive
- Increase compute intensity



Optimizations in Liger Kernel – Kernel Fusion



Optimizations in Liger Kernel – Benchmarks

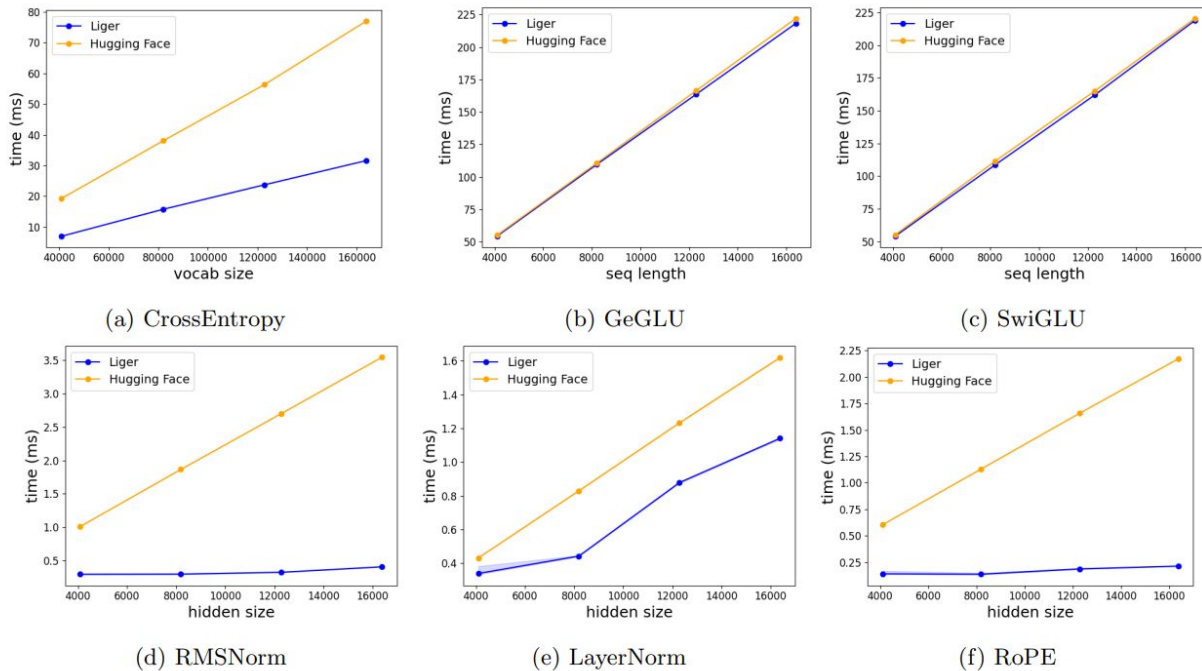


Figure 2: Kernel execution speed benchmarks.

Optimizations in Liger Kernel – Benchmarks

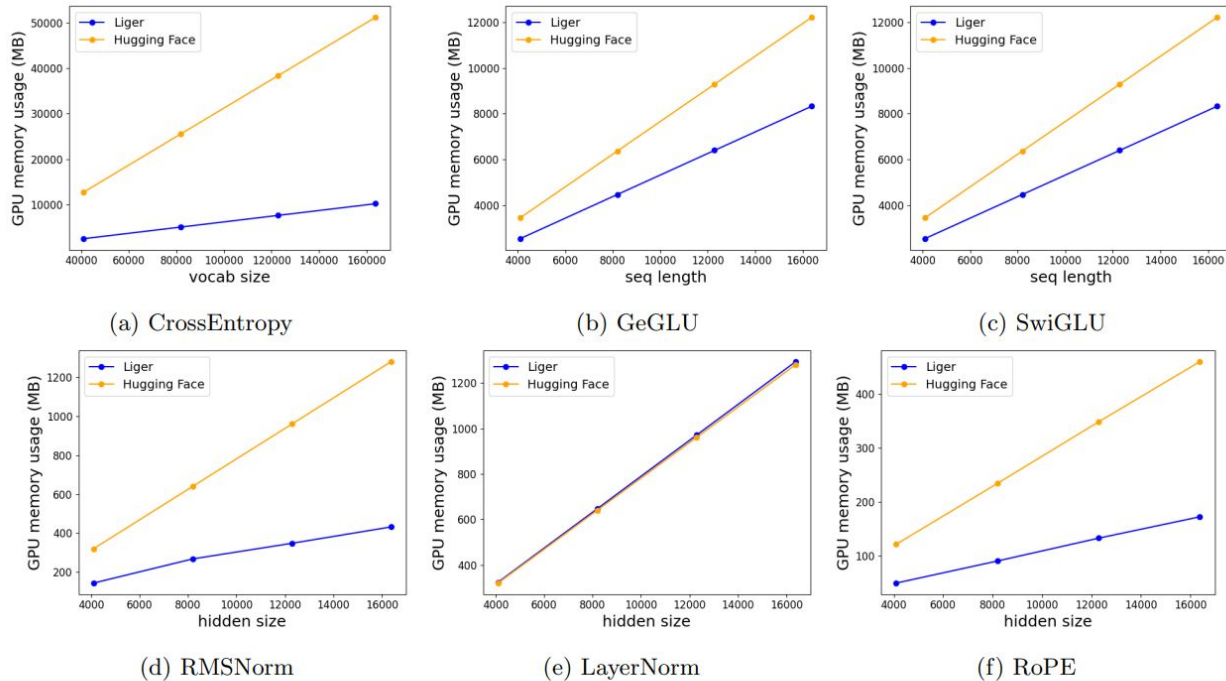


Figure 3: Kernel peak allocated memory benchmarks.

Optimizations in Liger Kernel – FusedLinearCrossEntropy

```
# Chunking loop
for chunk_id in range(num_chunks):
    # slicing tensors
    ...

    logits_chunk = _input_chunk @ weight.t() # chunk_size x V

    ...

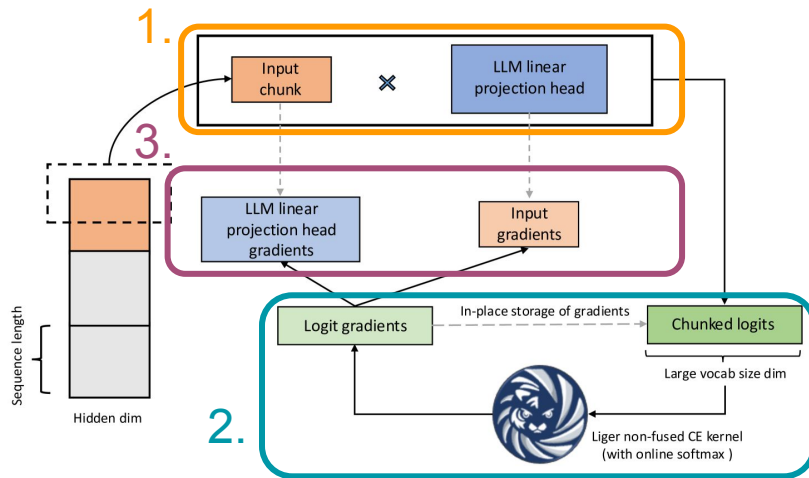
    # CE Kernel
    # Calculate the gradient of logits_chunk in place to save memory
    liger_cross_entropy_kernel[(n_rows,)](...)

    # Store output results
    ...

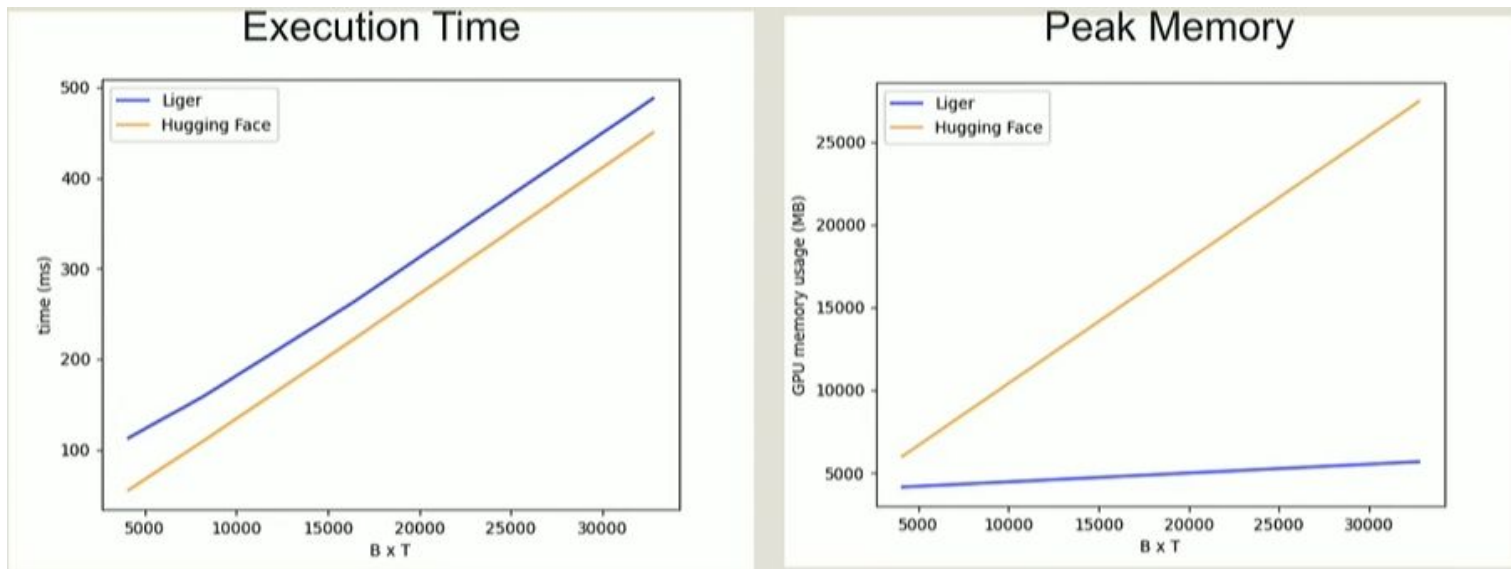
    # Backprop manually
    grad_logits_chunk = logits_chunk # chunk_size x V

    grad_input[start_idx:end_idx] = grad_logits_chunk @ weight

    if grad_weight is not None:
        torch.addmm(
            grad_weight,
            mat1=logits_chunk.t()
            mat2=_input_chunk,
            out=grad_weight,
        )
```



Optimizations in Liger Kernel – FusedLinearCrossEntropy



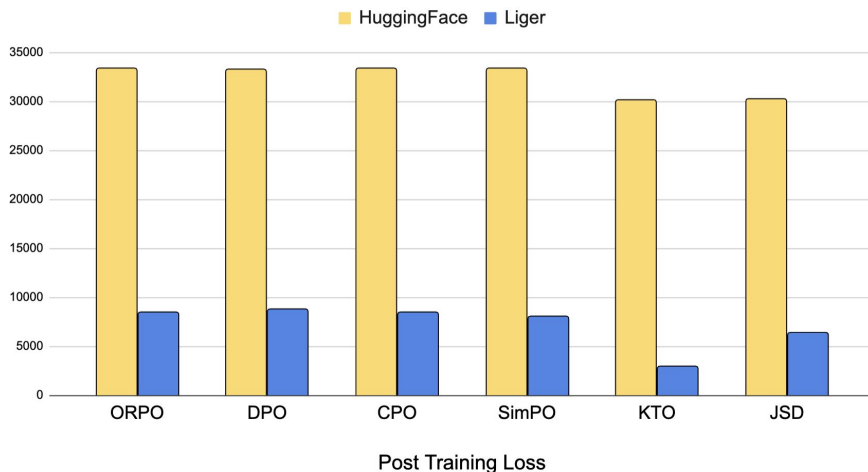
Chunked Loss – Post Training Kernels

80% more memory efficient post training losses (DPO, ORPO, CPO, etc)



```
from liger_kernel.chunked_loss import LigerFusedLinearDPOLoss
orpo_loss = LigerFusedLinearORPOLoss()
y = orpo_loss(lm_head.weight, x, target)
```

Peak Reserved Memory (MB)



Chunked Loss – Post Training Kernels

Integration in [huggingface/trl](https://huggingface.co/trl)

~80% memory reduction with native liger-kernel losses. For more information, see [linkedin/Liger-Kernel@ v0.5.0 \(release\)](#)

- ☐ DPO 🦙 [\[Liger\] liger DPO support #2568](#)
- ☐ BCO
- ☐ ORPO 🦙 [\[Liger\] add native liger-kernel orpo loss #2482](#)
- ☐ SimPO (in CPO) 🦙 [\[Liger\] Integrate Liger CPO & SimPO #2506](#)
- ☐ JSD (in GKD) 🦙 [\[WIP\] \[Liger\] liger JSD support #2573](#)
- ☒ GRPO 🦙 [Integrate Liger GRPO Loss to GRPO Trainer #3184](#)
- ☒ KTO 🦙 [\[Liger\] Liger KTO support #2812](#)

```
from trl import GRPOConfig

training_args = GRPOConfig(..., use_liger_kernel=True)
```

Testing in Liger Kernel

- Single Kernel
 - Correctness Test
 - Performance Test
- End-to-end
 - Convergence Test
 - Production Test

Testing in Liger Kernel – Correctness Test

- Why weird shapes?
 - Bug in LayerNorm kernel
 - [issue](#)
 - [pr](#)

```
# Using actual shapes from real world models
@pytest.mark.parametrize(
    "batch_size, seq_length, hidden_size",
    [
        (2, 8, 4096), # hidden_size of Llama
        (1, 1, 1023), # Minimal batch/seq with near power-of-2 hidden
        (3, 7, 256), # Prime numbers for batch/seq
    ],
)
@pytest.mark.parametrize(
    "dtype, atol, rtol",
    [
        # atol is for small values: they have more difference, so set atol higher
        # rtol is for larger values: they are very close, so set rtol lower
        (torch.float32, 1e-5, 1e-5),
        pytest.param(
            torch.bfloat16,
            1e-8,
            5e-2,
            marks=pytest.mark.skipif(not supports_bfloat16(), reason="bfloat16 not
supported on this GPU"),
        ),
    ],
)
```

Testing in Liger Kernel – Correctness Test

```

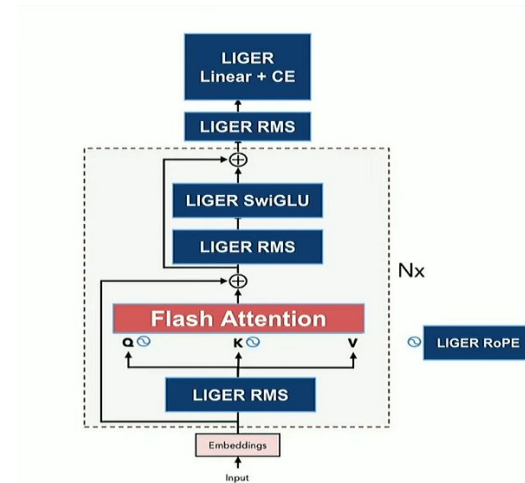
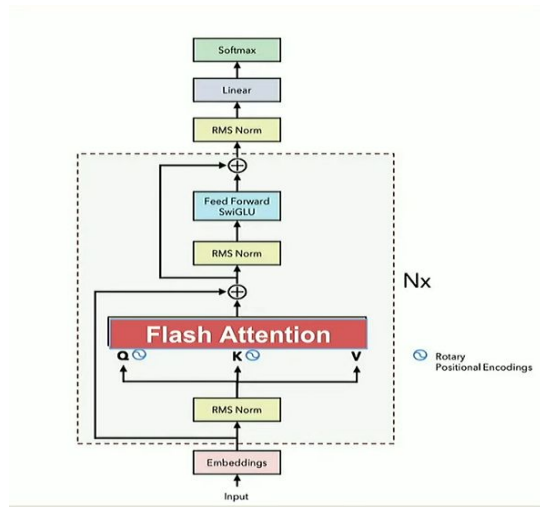
# Feature itself
@pytest.mark.parametrize("return_z_loss", [True, False])
@pytest.mark.parametrize(
    "lse_square_scale",
    [
        1e-4, # PaLM
        1e-5, # Chameleon
    ],
)
def test_correctness_with_z_loss_once(
```

```

# Feature combinations
@pytest.mark.parametrize(
    "return_z_loss, lse_square_scale",
    [
        (True, 1e-4),
        (False, 1e-5),
    ],
)
@pytest.mark.parametrize(
    "label_smoothing, ignore_index, reduction",
    [
        (0.1, 42, "mean"), # Non-default values
        (0.2, -42, "sum"),
    ],
)
def test_correctness_with_z_loss_with_other_params_once(
```

Testing in Liger Kernel – Convergence Test

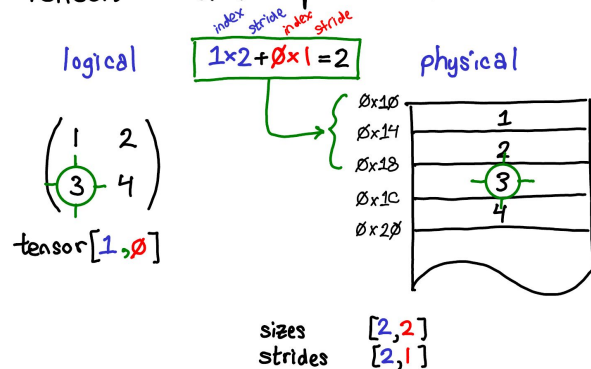
If every components are correct, what could possibly go wrong?



Contiguous

<https://ezyang.github.io/stride-visualizer/index.html>

Tensor: Strided Representation



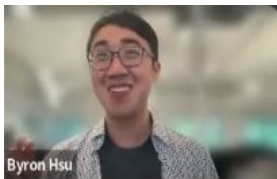
Upcoming: TensorAccessor

rev2

<https://blog.ezyang.com/2019/05/pytorch-internals/>

In-place tensor reuse

RMSNorm



[\[GPU MODE\] Lecture 28: Liger Kernel](#)

Yahh! Finished RMSNorm kernels!
RMSNorm goes brrrrr!!!



In-place tensor reuse

Issue: <https://github.com/linkedin/Liger-Kernel/issues/370>

PR: <https://github.com/linkedin/Liger-Kernel/pull/376>

Roadmap

- liger-kernel: [\[RFC\] Liger FlexChunkLoss: Alignment and Distillation loss](#)
 - trl: [\[Tracking issue\]](#) Integrate native liger-kernel losses
-
- Newer hardware specific instructions use
 - Tensor Parallel kernel support

Good first issue

- Improve documentation
- Support new models
- [Revert monkey patching](#)
- Enhance Benchmarking

Q&A